Università di Brescia

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Corso di Laurea in Ingegneria Informatica

Relazione del progetto di Robotica

Maze Exploration Search Analysis:

A Comparative Evaluation of A* and D*-Lite Replanning under Multi-Goal Detour-Index-Based Exploration

Anno di Corso 2025-2026

Docente del corso:

Prof. Enrico Scala

Prof. Luigi Gargioni

Studenti:

Francesco Guarda

Andrea Moro

Contents

1 Introduction

This project treats goal-directed maze exploration as an **online replanning problem** within the classical Micromouse paradigm. The agent knows the start and goal cells in advance but not the maze layout, and must interleave sensing, (re)planning, and acting to reach the goal while minimising exploration effort under **partial observability**.

This report evaluates two search algorithms, A* (full replanning) and D*-Lite (incremental replanning), isolating the effect of *how* a plan is repaired when a newly discovered wall invalidates the current plan. Both algorithms run against a common exploration framework built for this comparison: a shared interface allows either algorithm to drive the same sense-plan-act loop, while a headless extension of the MMS simulator makes fully automated, GUI-free evaluation at scale practical. This framework further extends the classical center-goal Micromouse problem to multi-goal exploration, maximizing exploration effort through goal placement via a **detour index**. A full experimental campaign then provides a controlled comparison of the two algorithms across a corpus of 55 mazes and a range of multi-goal scenarios.

2 Background

The Micromouse problem and the MMS simulator. The Micromouse competition tasks a small autonomous robot with exploring an unknown grid maze from a fixed start to a fixed goal region. This project uses `mms`[1], which reproduces that setting through a GUI for visualisation and a text-based `stdin`/`stdout` protocol for wall queries, moves, turns, and display commands. Its competition-faithful design, built-in visualisation, and existing Python bindings motivated its adoption here, where it is further extended in §3.

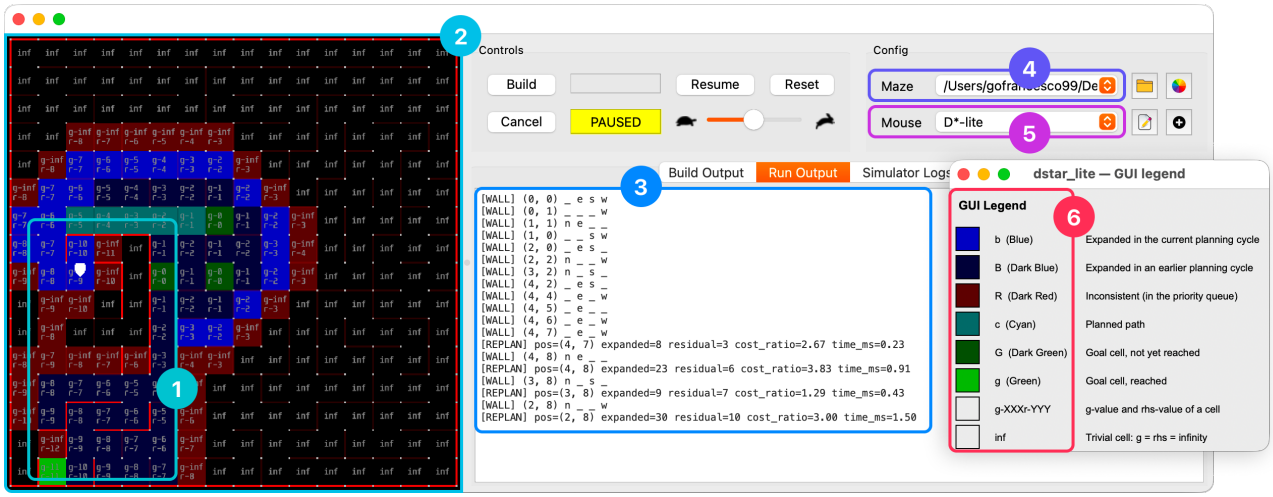

Figure 1: Annotated `mms` GUI during a running D*-Lite session: (1) wall segments and per-cell overlay text; (2) the maze grid; (3) the Run Output panel, logging sensing and replanning events; (4) the maze-selection path; (5) the algorithm-selection field; (6) the algorithm's GUI legend window.

Search under partial observability. Under the freespace assumption, an agent plans as though every unsensed cell were open, repairing its plan only when a sensed wall proves otherwise. This optimistic-planning principle underlies the D* family of incremental replanners [2]. The report compares two heuristic search strategies built on this principle: A*[3], the classical best-first baseline, and D*-Lite[4], designed explicitly to reuse prior search results rather than resolve the whole problem recursively from scratch.

3 Methodology

3.1 Shared Exploration Interface

`BaseAPI` is an abstract contract governing wall sensing, movement, display, and reset, decoupling an exploration algorithm from its I/O backend. `BaseAlgorithm`, in turn, implements the shared *sense* → *plan* → *act* → *log* loop on top of `BaseAPI`, along with goal resolution, accommodating the framework's diverse goal-placement options (Micromouse's default centre area, detour-index-based placement, explicit coordinates, or random placement), and the metrics hooks common to every search algorithm. Adding a third algorithm to the framework therefore requires implementing only a single class against this interface.

3.2 Headless execution for automated evaluation

A second backend, `SimAPI`, implements the same `BaseAPI` contract entirely in memory, without running the MMS GUI process, thereby enabling headless, non-visual simulation. This forms the primary foundation for large-scale deterministic experimental campaigns, which would otherwise be impractical under manual, GUI-based execution.

3.3 Exploration algorithms

A* treats every replanning event as an independent search. Whenever a newly discovered wall invalidates the current plan, it rebuilds an open and a closed list from scratch, rooted at the agent's current position, and expands nodes in order of increasing $f = g + h$ until a goal is popped. Nothing from a previous event carries over into the next. Movement cost is *uniform*, and in multi-goal exploration the algorithm greedily re-targets the nearest remaining goal once one is reached, discarding all information about the other goals as it restarts.

D*-Lite, by contrast, maintains two values per cell across the entire episode: `g(s)`, the current best-known cost from `s` to the goal, and `rhs(s)`, a one-step lookahead equal to the cheapest neighbour's `g`-value plus the corresponding movement cost. A cell is *consistent* once `g(s) = rhs(s)`, and the algorithm keeps every reachable cell consistent except where a change has just occurred. When a wall is newly confirmed, only the cells adjacent to that edge have their `rhs` recomputed and are inserted into the priority queue as inconsistent; repair then propagates outward from those cells until consistency is restored, touching only the region the wall affects. Everywhere else, `g` and `rhs` remain untouched and are reused as-is in the next planning cycle. Upon reaching a goal, the search retires it by setting its `rhs` to infinity and re-targets the next one without discarding this retained state.

Both algorithms' heuristics are fixed to *Manhattan distance*, which is both admissible and consistent and aligns with D*-Lite's own incremental `km`-based heuristic update. It further offers **constant-time complexity**, requiring no recomputation since it remains valid throughout the episode. This ensures that neither algorithm's time-related metrics are confounded by heuristic-recomputation overhead.

3.4 Multi-goal exploration

Given a set of goals, both algorithms visit them in greedy nearest-goal order, though the underlying search mechanics that produce this order differ. A*'s search terminates at the first goal popped from its open list, discarding all partial information about the remaining goals. D*-Lite, by contrast, roots its search in the *entire* remaining goal set simultaneously, initialising every unreached goal at `rhs = 0`; once the nearest goal is reached, the `g`/`rhs` state already built toward the others remains valid and is reused directly. This constitutes a multi-goal-specific amplification of the reuse advantage described above.

3.5 Modeling exploration effort: the detour index

For a reference cell and a candidate cell, the **detour index** is defined as the ratio between the true in-maze (BFS) distance and the Manhattan distance separating them. A cell that is topologically close yet remote within the maze thus yields a high detour index, rendering it highly deceptive and prone to misleading greedy planners operating under partial knowledge[5].

In the multi-goal, detour-index-based placement procedure, the per-cell detour index is first computed relative to the agent's starting position, and the first goal is placed at the cell maximising this index. At each subsequent step, the detour index is recomputed relative to the start and to every previously placed goal, with only the *minimum* value retained for each cell. This ensures that the selection of each new goal accounts for deceptiveness not only with respect to the start but also with respect to all previously fixed goal locations. Figure 2 illustrates the resulting per-cell detour index at each placement step k = 1..4 for a representative maze, showing how the retained-minimum criterion progressively reshapes the score map as goals accumulate.

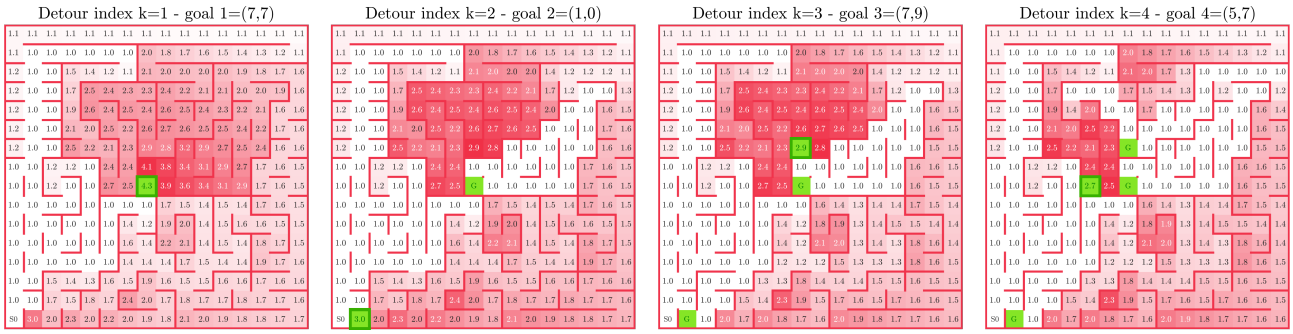

Figure 2: Score map maximised by goal placement at each step k = 1..4, for maze 2009japan.

This procedure yields a deterministic sequence of nested goals that are mutually misleading, thereby maximising exploration effort relative to one another, providing a controlled axis of *exploration effort*, which increases linearly with the amount k of placed goals.

4 Experimental Evaluation

4.1 Setup

The corpus comprises 55 standard Micromouse competition mazes[6], each 16×16 (256 cells), filtered to guarantee full connectivity from the start so that every goal is reachable and successful termination of exploration is thereby guaranteed. For each maze, four goal-count scenarios (k = 1..4) are placed automatically via detour-index placement (§3.5). The resulting headless batch campaign comprises **440 runs** in total, each logging run-level scalars together with a per-event replanning record, as detailed in the subsequent sections.

With the exception of wall-clock planning time, every metric is exactly reproducible across runs. The analysis presented below is therefore descriptive and paired, requiring no inferential statistics; planning time is the only quantity subject to genuine measurement noise.

4.2 Goal-placement across the maze corpus

Although detour-index placement progressively intensifies exploration effort as k grows, Table 1 spans the resulting range of first-goal placements across a representative sample of the maze corpus and reveals that a high detour score can arise from two qualitatively different situations: goals that are genuinely remote *and* high-scoring, and goals whose high score reflects only their proximity to the start

rather than any real difficulty. This start-proximity pattern recurs in 26 of the 55 mazes; Figure 2 illustrates one such instance, where the second placed goal, as opposed to the genuinely remote others, sits immediately behind a wall close to the reference set.

Maze	Manhattan dist.	Real (BFS) dist.	Detour score
84japx	1	7	7.0
2008japan	11	71	6.5
museum	1	5	5.0
2009japan	14	60	4.3

Table 1: First-goal (k=1) placement for four mazes: `84japx`, `2008japan`, `2009japan`, and `museum`; the second and last are genuinely remote, while first and third score highly because of start proximity.

4.3 Replanning cost: nodes expanded and wall-clock planning time

D*-Lite consistently expands between roughly two and three times fewer nodes than A*, and accumulates a correspondingly lower cumulative planning time, on the order of two-and-a-half to three times less (Figure 3). D*-Lite expands fewer nodes in 212 of the 220 matched (maze, k) pairs and achieves lower planning time in 219 of the 220 pairs. This outcome follows directly from the incremental-repair mechanism described in §3.3: because only the cells affected by a newly discovered wall are reprocessed, computational cost tracks the magnitude of the change rather than the size of the remaining problem as a whole.

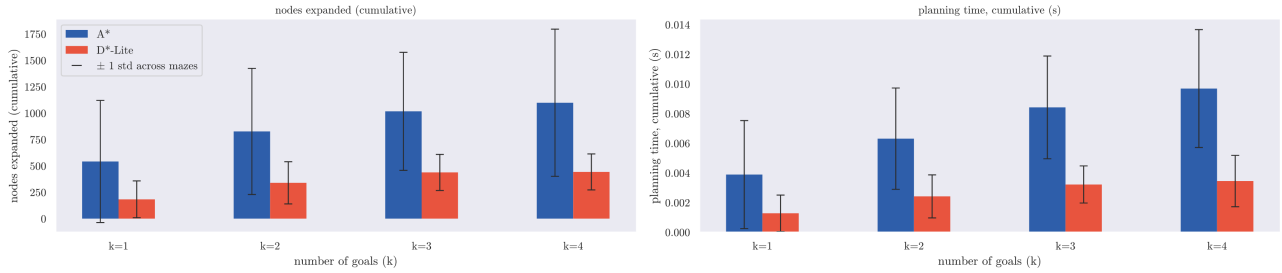

Figure 3: Cumulative nodes expanded and cumulative planning time, A* vs. D*-Lite, grouped by goal-count scenario (mean across the 55 mazes, ±1 std-dev error bars).

4.4 Search-cost scaling: evidence for incremental reuse

A*'s per-event search cost scales more steeply with residual distance to the goal than does D*-Lite's, and this gap widens as k increases. Within the dense band of residual distances up to 30 cells, a linear regression of nodes expanded on residual distance shows A*'s slope rising from approximately 1.8 at k = 1 to nearly 2.8 at k = 4, whereas D*-Lite's remains comparatively flat.

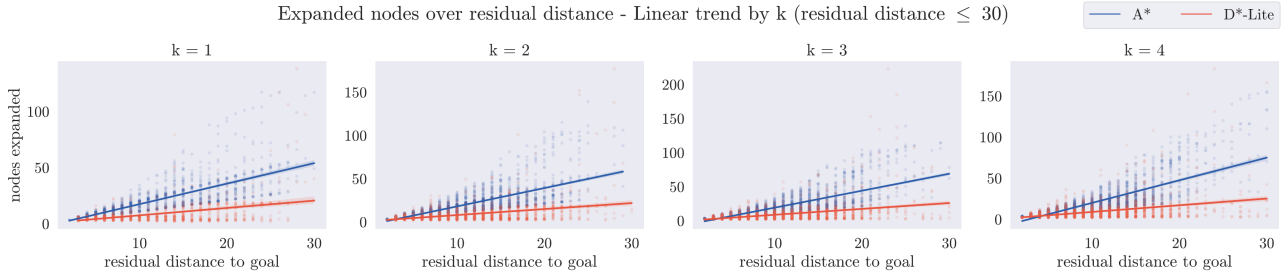

Figure 4: Nodes expanded as a function of residual distance to the goal, per replanning event, faceted by goal-count scenario (k = 1..4), with linear trend and 95% confidence band.

Pooled across all values of k, A*'s slope is roughly three times steeper than D*-Lite's (Figure 4). This provides direct empirical support for D*-Lite's central claim (§3.3): repair cost scales with the *size of the region affected by a change*, rather than with the size of the whole remaining problem, whereas A*'s from-scratch cost grows with the full remaining search space.

4.5 Memory footprint of search structures

The two algorithms make opposite trade-offs between search cost and memory (Figure 5). In a single run of maze 00japan at k = 4, A*'s combined open and closed set oscillates within a low band, resetting at every replanning event. D*-Lite's working set, by contrast, climbs toward a plateau near 80% of the maze and remains there. This climb is not perfectly monotonic, newly discovered walls may reset cells g values to infinity, but the overall trajectory is unmistakably one of accumulation rather than reset. The pooled trend (Figure 5, right) confirms that this pattern holds across the corpus: D*-Lite's occupancy rises steadily with event count, while A*'s remains essentially flat.

Table 2 summarises this pattern across all runs. D*-Lite's median peak occupancy is more than double A*'s (e.g., 182.5 vs. 69.0) and its median per-run mean occupancy is roughly four times higher (e.g., 116.58 vs. 29.51). No A* run's peak occupancy exceeds 75% of the maze, whereas 31 of the 220 D*-Lite runs exceed 90%, and four fill the maze entirely. This reverses the ordering established in §4.3: A* achieves its bounded memory footprint at the cost of repeated search, whereas D*-Lite achieves cheap repair at the cost of retained state. Neither algorithm dominates outright.

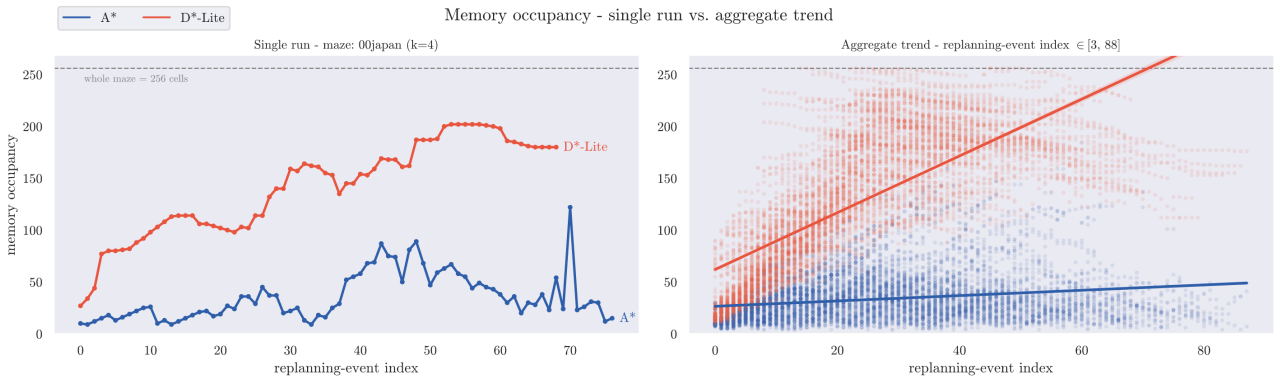

Figure 5: Memory occupancy of search structures over successive replanning events. Left: a single run (maze 00japan, k=4). Right: the same trend pooled across all runs.

Algorithm	Peak — median	Peak — σ	Mean — median	Mean — σ
A*	69.0	38.76	29.51	10.35
D*-Lite	182.5	71.63	116.58	47.21

Table 2: Peak and per-run mean memory occupancy (cells, out of 256), median and standard deviation across all runs.

5 Conclusions and Future Work

On the metrics that matter most, D*-Lite's incremental repair consistently outperforms A*'s from-scratch replanning: it expands fewer nodes per event and accumulates less planning time overall, with the gap widening as replanning distance increases. This advantage, however, comes at the cost of memory. Neither algorithm dominates outright; the preferable choice depends on which resource is scarcer, and on micromouse-class embedded hardware, this is not a foregone conclusion.

The main limitation of this evaluation is that the detour index recurrently biases goal placement

toward the start rather than calibrating absolute exploration effort. Introducing a start-proximity correction to the metric would represent the most direct next step. A second promising direction is to extend the framework to additional algorithms, weighted and anytime variants in particular, since incorporating a new algorithm requires implementing only a single class against this interface (§3.1). The full implementation, maze corpus, and experimental logs are available in the project repository[7].

References

1. **MMS simulator:** mackorone, *mms — A Micromouse Simulator*, v1.2.0, MIT License, GitHub (2024).
2. **D* / the freespace assumption:** A. Stentz, "Optimal and Efficient Path Planning for Partially-Known Environments," *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 1994.
3. **A*:** P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107, 1968.
4. **D*-Lite:** S. Koenig and M. Likhachev, "D* Lite," *Proc. AAAI/IAAI*, 476–483, 2002; and S. Koenig and M. Likhachev, "Fast Replanning for Navigation in Unknown Terrain," *IEEE Transactions on Robotics*, 21(3), 354–363, 2005.
5. **Detour index / route factor:** M. Barthélemy, "Spatial Networks," *Physics Reports*, 499(1–3), 1–101, 2011; M. T. Gastner and M. E. J. Newman, "The Spatial Structure of Networks," *European Physical Journal B*, 49(2), 247–252, 2006.
6. **Maze corpus:** J. Weisberg, *Micromouse Maze Collection*, tcp4me.com.
7. **Rob-26-MazeSolver repository:** F. Guarda and A. Moro, *Robotica 2026 - Maze Solver Project*, software, MIT License, GitHub (released 2026-07-29).

Acknowledgements. The authors acknowledge the use of large language model (LLM)-based artificial intelligence tools during the preparation of this report. These tools were used exclusively to improve the clarity and readability of the manuscript and to assist with source code development and debugging. All technical content, implementation decisions, experimental methodology, and conclusions remain the sole responsibility of the authors.